

Web Design (Basics of HTML, CSS, and JavaScript) -
Understanding the web content delivery, Understanding HTML and XHTML Connections,
Understanding Cascading Style Sheets, Understanding JavaScript

Web Design Basics

1. What is Web Design?

Web design is the process of **creating and designing websites** that are visually appealing, user-friendly, and functional. It includes three main technologies:

1. **HTML (HyperText Markup Language)** → Provides the structure of a webpage.
2. **CSS (Cascading Style Sheets)** → Styles and designs the webpage.
3. **JavaScript** → Adds interactivity and dynamic behavior.

A good web design ensures a smooth user experience, fast performance, and responsiveness across different devices.

Web Design Best Practices

- ✓ **Keep it simple and clean** → Avoid clutter.
- ✓ **Use readable fonts and colors** → Ensure good contrast.
- ✓ **Make it mobile-friendly** → Use responsive design.
- ✓ **Optimize images** → Reduce load time.
- ✓ **Test on different browsers** → Ensure compatibility.

Web Content Delivery

Web content delivery is the process of serving web pages and multimedia content to users via the internet.

Key Components of Web Content Delivery:

1. **Web Server:** Stores and serves website files.
 - Example: Apache, Nginx, IIS.
 - When you type `www.example.com`, the browser requests the webpage from the web server.
2. **Client (Web Browser):** The software that displays web pages.
 - Example: Chrome, Firefox, Edge.

- When you open `https://google.com`, your browser sends a request to Google's server.
 - 3. **HTTP/HTTPS (Protocols for Communication):**
 - **HTTP (HyperText Transfer Protocol):** Transfers web pages without encryption.
 - **HTTPS (Secure HTTP):** Uses encryption (SSL/TLS) for security.
 - Example: `http://example.com` (Unsecured) vs. `https://example.com` (Secured).
 - 4. **DNS (Domain Name System):** Converts domain names into IP addresses.
 - Example: `www.google.com` → `142.250.182.206`.
 - 5. **CDN (Content Delivery Network):** A network of distributed servers that deliver web content faster.
 - Example: Cloudflare, Akamai, AWS CloudFront.
 - YouTube uses CDNs to load videos faster from servers closer to the user.
-

2. HTML & XHTML Connections

HTML (HyperText Markup Language)

HTML is used to create the structure of web pages using tags.

Example of a Simple HTML Page:

```
<!DOCTYPE html>

<html>

<head>

    <title>My First Web Page</title>

</head>

<body>

    <h1>Welcome to My Website</h1>

    <p>This is a paragraph of text.</p>

    

</body>
```

```
</html>
```

XHTML (Extensible HyperText Markup Language)

XHTML is a stricter and cleaner version of HTML that follows XML rules.

Example of an XHTML Page:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

<head>

    <title>XHTML Example</title>

</head>

<body>

    <h1>XHTML Page</h1>

    <p>This is a well-structured XHTML document.</p>

    

</body>

</html>
```

Key Differences Between HTML & XHTML

Feature	HTML	XHTML
Syntax	Flexible	Strict
Tag Closure	Optional	Mandatory (<code>
</code> instead of <code>
</code>)
Nesting	Less strict	Must be properly nested
Case Sensitivity	Not case-sensitive	Lowercase required (<code><title></code> , not <code><TITLE></code>)
Compatibility	Older browsers support it	Requires modern browsers

- **Web Content Delivery** ensures fast and secure webpage loading using servers, browsers, HTTP/HTTPS, DNS, and CDNs.
- **HTML** is the standard language for creating webpages, while **XHTML** follows stricter syntax for better compatibility and structure.

HTML (HyperText Markup Language) - Basics

1. What is HTML?

HTML (**HyperText Markup Language**) is the **foundation of web development**, used to create and structure webpages. It consists of **elements (tags)** that define content like headings, paragraphs, images, and links.

Key Features:

- ✓ Defines the **structure** of a webpage.
- ✓ Uses **tags** enclosed in angle brackets (`<` `>`).
- ✓ Works with **CSS for styling** and **JavaScript for interactivity**.

2. Basic HTML Structure

Every HTML document follows a basic structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>My First Web Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>Welcome to my first webpage.</p>
</body>
</html>
```

Explanation of Tags:

Tag	Description
<code><!DOCTYPE html></code>	Declares the document type (HTML5).
<code><html></code>	Root element of an HTML page.
<code><head></code>	Contains metadata (not visible on the page).
<code><title></code>	Sets the page title (shown on the browser tab).
<code><meta charset="UTF-8"></code>	Supports special characters (like emojis 🌸).
<code><meta name="viewport"></code>	Ensures responsiveness on mobile devices.
<code><body></code>	Contains the visible content of the webpage.

3. Common HTML Tags

Headings (<h1> to <h6>)

```
<h1>Main Heading</h1>
<h2>Subheading</h2>
<h3>Smaller Heading</h3>
```

- <h1> is the largest, <h6> is the smallest.

Paragraphs and Line Breaks

```
<p>This is a paragraph.</p>
<p>This is another paragraph.</p>
<br>
<p>Line break above!</p>
```

- <p> creates a new paragraph.
-
 inserts a line break.

Links (<a> - Anchor Tag)

```
<a href="https://www.google.com">Visit Google</a>
```

- href="URL" → Specifies the link destination.

Images ()

```

```

- src="image.jpg" → Image source (URL or file path).
- alt="Description" → Alternative text for accessibility.
- width="300" → Sets image width (height adjusts automatically).

Lists (Ordered & Unordered)

Unordered List ()

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Cherry</li>
</ul>
```

Displays items with bullet points.

Ordered List ()

```
<ol>
  <li>Step 1</li>
  <li>Step 2</li>
  <li>Step 3</li>
</ol>
```

Displays items with numbers (1, 2, 3...).

Tables (<table>)

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Alice</td>
    <td>22</td>
  </tr>
  <tr>
    <td>Bob</td>
    <td>25</td>
  </tr>
```

```
</table>
```

Name	Age
Alice	22
Bob	25

- `<table>` → Creates a table.
 - `<tr>` → Table row.
 - `<th>` → Table header (bold).
 - `<td>` → Table data (normal cell).
-

4. Forms (User Input)

Forms allow users to enter data, such as login details.

```
<form>
  <label for="name">Name:</label>
  <input type="text" id="name" placeholder="Enter your name">

  <label for="email">Email:</label>
  <input type="email" id="email" placeholder="Enter your email">

  <label for="password">Password:</label>
  <input type="password" id="password">

  <input type="submit" value="Submit">
</form>
```

Common Form Elements:

Input Type	Description
<code>text</code>	Plain text input
<code>email</code>	Email validation

`password` Password input

`submit` Submit button

5. HTML Comments

```
<!-- This is a comment -->
<p>This is visible text.</p>
```

✓ Comments help explain the code but are **not visible** on the webpage.

6. HTML Attributes

Attributes **add extra information** to elements.

```
<a href="https://example.com" target="_blank">Open in New Tab</a>
```

`target="_blank"` → Opens link in a new tab.

7. HTML Page Layout

```
<header>
  <h1>Website Header</h1>
</header>

<nav>
  <a href="#">Home</a> | <a href="#">About</a> | <a href="#">Contact</a>
</nav>

<main>
  <article>
    <h2>Article Title</h2>
    <p>Article content goes here...</p>
  </article>
</main>

<footer>
  <p>&copy; 2025 My Website</p>
</footer>
```

Semantic Elements for Layout:

Element	Purpose
<header>	Website header (logo, title, etc.)
<nav>	Navigation links
<main>	Main content of the page
<article>	Standalone content (blog post, article)
<footer>	Footer (copyright, contact info)

Summary Table

Feature	HTML Tag	Example
Heading	<code><h1></code> to <code><h6></code>	<code><h1>Title</h1></code>
Paragraph	<code><p></code>	<code><p>Text here</p></code>
Link	<code><a></code>	<code>Click</code>
Image	<code></code>	<code></code>
List	<code></code> / <code></code>	<code>Item</code>
Table	<code><table></code>	<code><table><tr><td>Data</td></tr></table></code>
Form	<code><form></code>	<code><input type="text"></code>

HTML provides the basic structure of a webpage.
It works with CSS (for design) and JavaScript (for interactivity).
Practice by creating simple webpages!

CSS (Cascading Style Sheets) - Basics

1. What is CSS?

CSS (**Cascading Style Sheets**) is used to **style** HTML elements, making webpages look **attractive and user-friendly**. It controls the **colors, fonts, layouts, spacing, and animations** of a webpage.

Key Features of CSS:

- ✓ **Separates design from HTML structure** (cleaner code).
- ✓ **Applies styles to multiple pages** (via external stylesheets).
- ✓ **Responsive design** (works on all screen sizes).

2. Ways to Apply CSS

There are **three** ways to use CSS in an HTML document:

1 Inline CSS (Inside HTML Tags)

Applied directly to an element using the `style` attribute.

```
<p style="color: blue; font-size: 18px;">      This is a blue  
paragraph.</p>
```

Pros: Quick styling.

Cons: Not reusable, difficult to maintain.

2 Internal CSS (Inside `<style>` in `<head>`)

Defined within the `<style>` tag in the HTML document.

```
<!DOCTYPE html>  
<html>  
<head>  
  <style>  
    body {  
      background-color: lightgray;  
    }  
    h1 {  
      color: red;  
    }  
  </style>  
</head>  
<body>  
  <h1>Red Heading</h1>  
</body>  
</html>
```

Pros: Works for a single page, avoids repetition.

Cons: Not reusable across multiple pages.

3 External CSS (Recommended - Separate `.css` File)

Best practice! Saves styles in an external file and links it to HTML.

Step 1: Create a CSS file (styles.css)

```
body {  
    background-color: #f0f0f0;  
    font-family: Arial, sans-serif;  
}  
h1 {  
    color: blue;  
    text-align: center;  
}
```

Step 2: Link CSS in HTML

```
<link rel="stylesheet" href="styles.css">
```

Pros: Reusable across multiple pages.

Cons: Needs an extra file.

3. CSS Selectors

CSS **selectors** target HTML elements and apply styles.

❶ Universal Selector (*)

Applies styles to **all** elements.

```
* {  
    margin: 0;  
    padding: 0;  
}
```

❷ Element Selector

Targets **specific HTML tags**.

```
h1 {  
    color: green;  
}  
p {  
    font-size: 18px;  
}
```

③ Class Selector (.)

Targets elements with a specific **class**.

```
.special-text {  
    color: purple;  
    font-weight: bold;  
}
```

Usage in HTML:

```
<p class="special-text">This is styled using a class.</p>
```

④ ID Selector (#)

Targets a **single, unique element**.

```
#main-title {  
    font-size: 30px;  
    color: darkblue;  
}
```

Usage in HTML:

```
<h1 id="main-title">Main Title</h1>
```

5 Grouping Selector

Applies styles to multiple elements at once.

```
h1, h2, p {  
    color: navy;  
}
```

4. CSS Properties & Examples

1 Colors

Named Colors:

```
p {  
    color: red;  
}
```

Hexadecimal Colors (#rrggbb):

```
h1 {  
    color: #ff5733;  
}
```

RGB Colors:

```
body {  
    background-color: rgb(240, 240, 240);  
}
```

2 Background Styling

```
body {
```

```
background-color: lightblue;
background-image: url("background.jpg");
background-size: cover;
background-position: center;
}
```

3 Text Styling

```
p {
  font-family: Arial, sans-serif;
  font-size: 18px;
  font-weight: bold;
  text-align: center;
  text-decoration: underline;
  text-transform: uppercase;
}
```

4 Box Model (Margin, Padding, Border)

Box Model = Content + Padding + Border + Margin

```
.box {
  width: 300px;
  height: 200px;
  background-color: lightgray;
  padding: 20px;
  border: 2px solid black;
  margin: 30px auto;
}
```

Margin = Space outside the element

Padding = Space inside the element

5 CSS Layout - Display & Positioning

Block vs. Inline Elements

- **Block elements** (`div`, `p`, `h1`) take up the full width.
- **Inline elements** (`span`, `a`) only take up the necessary space.

Positioning

```
.fixed-box {  
  position: fixed;  
  top: 10px;  
  right: 20px;  
}
```

Position	Description
<code>static</code>	Default (normal position).
<code>relative</code>	Moves relative to itself.
<code>absolute</code>	Moves relative to the nearest positioned ancestor.
<code>fixed</code>	Stays in place while scrolling.

5. Responsive Design (Media Queries)

Make websites **mobile-friendly** using media queries.

```
@media (max-width: 600px) {  
  body {  
    background-color: yellow;  
  }  
}
```

This will change the background color to **yellow** if the screen width is **600px or smaller**.

6. CSS Animations

```
@keyframes slide {  
  0% { transform: translateX(0); }  
  100% { transform: translateX(100px); }  
}  
  
.box {  
  width: 100px;  
  height: 100px;  
  background-color: red;  
  animation: slide 2s infinite alternate;  
}
```

Moves a box **left to right** infinitely.

7. Simple CSS Practice Project

1 Create an `index.html` File

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width,  
initial-scale=1.0">  
  <title>My First Styled Page</title>  
  <link rel="stylesheet" href="style.css">  
</head>  
<body>  
  <h1 class="title">Welcome to My Website</h1>  
  <p>This is a simple webpage styled using CSS.</p>  
  <button class="btn">Click Me</button>
```

```
</body>
</html>
```

2 Create a **style.css** File

```
body {
    background-color: #f0f0f0;
    text-align: center;
    font-family: Arial, sans-serif;
}

.title {
    color: darkblue;
    font-size: 30px;
    text-transform: uppercase;
}

p {
    font-size: 18px;
    color: gray;
}

.btn {
    background-color: blue;
    color: white;
    padding: 10px 20px;
    border: none;
    cursor: pointer;
    font-size: 16px;
}

.btn:hover {
    background-color: darkblue;
}
```

☐ Save both files and open **index.html** in a browser.

Try modifying the styles and see what changes!

Summary

- ✓ CSS makes webpages visually appealing.
- ✓ It can be applied inline, internally, or externally (best practice).
- ✓ Selectors target elements for styling.
- ✓ The box model controls spacing and layout.
- ✓ Media queries enable responsiveness for mobile devices.
- ✓ Animations add interactivity.

JavaScript Basics for Beginners

JavaScript (JS) is a **programming language** used to add **interactivity, dynamic behavior, and logic** to web pages. It works alongside **HTML and CSS** to create fully functional websites.

Why Learn JavaScript?

- ✓ Adds interactivity (buttons, forms, animations)
- ✓ Runs in all web browsers (no installation needed)
- ✓ Supports modern frameworks (React, Vue, Angular)

1. How to Use JavaScript in HTML

There are three ways to include JavaScript in an HTML file:

1 Inline JavaScript

JavaScript code is written **inside HTML elements** using the `onclick` or other event attributes.

```
<button onclick="alert('Hello, World!')">Click Me</button>
```

Not recommended for large projects.

2 Internal JavaScript (<script> in HTML)

JS is placed inside a `<script>` tag within the HTML file.

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>JavaScript Example</title>
</head>
<body>
  <script>
    console.log("Hello from JavaScript!");
  </script>
</body>
</html>
```

③ External JavaScript (Recommended)

JavaScript code is stored in a **separate file** (`script.js`) and linked to HTML.

Step 1: Create a JS File (`script.js`)

```
console.log("Hello, this is an external JavaScript file!");
```

Step 2: Link It in HTML

```
<script src="script.js"></script>
```

Best practice: Keeps HTML clean and JavaScript reusable.

2. JavaScript Basics

① Variables (`var`, `let`, `const`)

Variables store data values.

```
var name = "Alice"; // Old way (avoid using)
let age = 25;        // Can be changed later
const country = "India"; // Cannot be changed
console.log(name, age, country);
```

Use **let** and **const** instead of **var**.

2 Data Types

JavaScript has different data types:

```
let num = 10;           // Number
let text = "Hello";     // String
let isTrue = true;      // Boolean
let numbers = [1, 2, 3]; // Array
let person = {name: "Alice", age: 25}; // Object
```

3 Operators

JavaScript supports arithmetic and comparison operators.

```
let a = 10, b = 5;
console.log(a + b); // Addition (15)
console.log(a * b); // Multiplication (50)
console.log(a > b); // Comparison (true)
console.log(a === 10); // Strict equality (true)
```

4 Functions

Functions help reuse code.

```
function greet(name) {
    return "Hello, " + name + "!";
}
console.log(greet("Alice"));
```

Arrow Function (Shorter Version)

```
const add = (x, y) => x + y;
console.log(add(5, 3)); // 8
```

5 Conditional Statements (if-else)

```
let score = 85;
if (score > 80) {
  console.log("Great job!");
} else {
  console.log("Keep trying!");
}
```

6 Loops (for, while)

Loops help repeat tasks.

```
for (let i = 1; i <= 5; i++) {
  console.log("Number: " + i);
}

let j = 1;
while (j <= 3) {
  console.log("Count: " + j);
  j++;
}
```

3. DOM Manipulation (Changing Web Pages)

The **DOM (Document Object Model)** allows JavaScript to change HTML & CSS dynamically.

1 Selecting Elements

```
let title = document.getElementById("heading"); // Select by ID
```

```
let paragraphs = document.getElementsByTagName("p"); // Select all <p>
elements
let buttons = document.querySelector(".btn"); // Select by class
```

2 Changing Text

```
document.getElementById("heading").innerText = "Hello, JavaScript!";
```

3 Changing Styles

```
document.getElementById("heading").style.color = "blue";
```

4 Event Handling (Click Button)

```
document.getElementById("myButton").addEventListener("click",
function() {
    alert("Button Clicked!");
});
```

Use this method instead of **onclick** in HTML.

4. JavaScript Example: Interactive Web Page

1 Create `index.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Example</title>
    <style>
        body { text-align: center; font-family: Arial; }
        #box { width: 100px; height: 100px; background: red; margin:
auto; }
```



```
        </style>
</head>
<body>
    <h1 id="heading">Welcome!</h1>
    <p>Click the button to change the box color.</p>
    <div id="box"></div>
    <button id="myButton">Change Color</button>

    <script src="script.js"></script>
</body>
</html>
```

2 Create `script.js`

```
document.getElementById("myButton").addEventListener("click",
function() {
    let box = document.getElementById("box");
    let colors = ["red", "blue", "green", "yellow", "purple"];
    let randomColor = colors[Math.floor(Math.random() *
colors.length)];
    box.style.backgroundColor = randomColor;
});
```

Open `index.html` in a browser and click the button!

Summary

- ✓ JavaScript adds interactivity to websites.
- ✓ Use `let` and `const` for variables.
- ✓ Use `if-else`, `for`, `while` to control logic.
- ✓ Modify web pages using the DOM.
- ✓ Use event listeners to handle user actions.